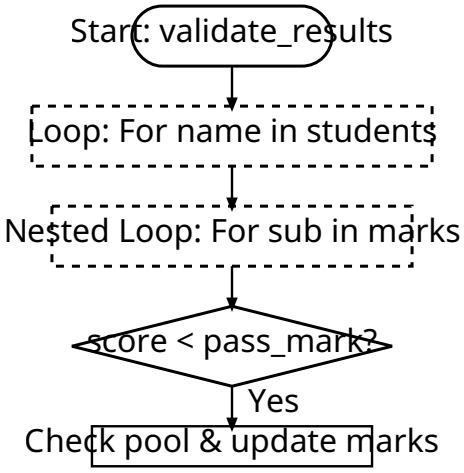
CURRICULUM TRACE REPORT: CODE ANALYSIS & TRACE

1. Code Block 1: validate_results

Output: `{'John': {'Math': 35, 'Science': 45, 'History': 78}, 'status': 'Pass'}, 'Mary': {'Math': 33, 'Science': 30, 'History': 85}, 'status': 'Fail (2 subjects)'}`

```
def validate_results(students, pass_mark, max_mark, grace_pool):
    results = {}
    for name in students:
        marks = students[name]
        updated_marks = {}
        pool_left = grace_pool
        failed_count = 0
        for sub in marks:
            score = marks[sub]
            if score < pass_mark:
                needed = pass_mark - score
                if pool_left >= needed:
                    pool_left -= needed
                    score = pass_mark
            if score > max_mark: score = max_mark
            if score < pass_mark: failed_count += 1
            updated_marks[sub] = score
        if failed_count == 0: status = "Pass"
        else: status = "Fail (" + str(failed_count) + " subjects)"
        results[name] = {"marks": updated_marks, "status": status}
    return results
```

Flowchart Matrix



Detailed Curriculum Concepts

- **Computational Thinking:** Decomposes student validation into individual subject scoring loops, evaluating and mitigating passing thresholds using a shared pool.
- **Operators & Types:** Uses comparison (<, >=), arithmetic operators (-, -=), and string concatenation (+). Maps dictionary and string structures to integer modifications.
- **Tuples & Unpacking:** Dynamic key-based updates via students[name] iteration instead of direct multi-variable item unpacking.
- **Strings:** Dynamically builds status labels using structural casting with str(failed_count) combined via concatenation.
- **Code Analysis:** Evaluates individual local failure scopes. If a student needs more grace marks than remain in pool_left, the code leaves the score untouched and increments failed_count.
- **Algorithms & Pseudocode:** For each student, initialize local parameters. For each sub, if failing and grace budget allows, consume budget and update to pass_mark. Count remaining failures.
- **Pattern Generation & Loops:** Employs a nested iteration configuration where the outer sequence processes entities and the inner sequence updates records.
- **Nested Lists & Tuples:** Operates on nested object abstractions (Dict[str, Dict[str, int]]) reflecting multi-tiered grade structures.

Code Tracing Execution Log

Student	Subject	Initial Score	Condition & Action	Pool Left	Failed Count	Updated Score
John	Math	32	32 < 35 (Needs 3). Pool 5 >= 3. Deduct 3. Passed.	2	0	35
John	Science	45	45 >= 35. No change.	2	0	45
John	History	78	78 >= 35. No change.	2	0	78
Mary	Math	28	28 < 35 (Needs 7). Pool 2 < 7. No grace applied.	2	1	28
Mary	Science	30	30 < 35 (Needs 5). Pool 2 < 5. No grace applied.	2	2	30
Mary	History	85	85 >= 35. No change.	2	2	85

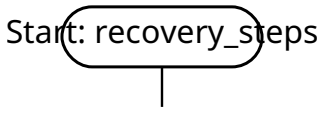
---

2. Code Block 2: recovery_steps

Output: `{'current': 0.6, 'consecutive_needed': 12}`

```
def recovery_steps(attended, total, threshold=0.75):
    current = attended / total
    needed = 0
    while (attended + needed) / (total + needed) < threshold:
        needed += 1
    return {"current": current, "consecutive_needed": needed}
```

Flowchart Matrix



Detailed Curriculum Concepts

- **Computational Thinking:** Implements a deterministic evaluation system calculating distance from target attendance thresholds.
- **Operators & Types:** Employs division (/) resulting in floats, alongside sequential increment operations (+=).
- **Tuples & Unpacking:** Scalar parameter evaluation without iterative container breakdown steps.
- **Strings:** Standard internal key configuration containing no variable string modifications.
- **Code Analysis:** Tracks edge boundaries where total elements scale step-by-step alongside active counts until the percentage threshold criteria is met.
- **Algorithms & Pseudocode:** While current ratio remains below target, systematically simulate additional perfect tracking entries.
- **Pattern Generation & Loops:** Uses a conditional while structure which repeats continuously based on variable states.
- **Nested Lists & Tuples:** Single tier numerical tracking layout without nested collections.

Code Tracing Execution Log

Iteration	Simulated Attended (12 + needed)	Simulated Total (20 + needed)	Current Ratio	Target Threshold	Condition (Ratio < Target)	Next Needed
Initial	12	20	0.6000	0.75	True	1
1	13	21	0.6190	0.75	True	2
4	16	24	0.6667	0.75	True	5
8	20	28	0.7143	0.75	True	9
11	23	31	0.7419	0.75	True	12
12	24	32	0.7500	0.75	False (Loop Stops)	12 (Final)

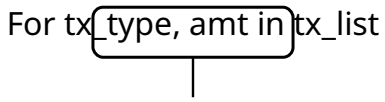
---

3. Code Block 3: simulate_atm

Output: `{'balance': 8000, 'history': ['withdraw: Success', 'deposit: Success']}`

```
def simulate_atm(balance, tx_list, daily_limit=50000):
    total_withdrawn = 0
    summary = []
    for tx_type, amt in tx_list:
        if amt <= 0: status = "Rejected"
        elif tx_type == "deposit":
            balance += amt
            status = "Success"
        elif tx_type == "withdraw":
            if amt > balance or total_withdrawn + amt > daily_limit: status =
"Rejected"
            else:
                balance -= amt
                total_withdrawn += amt
                status = "Success"
        summary.append(tx_type + ": " + status)
    return {"balance": balance, "history": summary}
```

Flowchart Matrix



Detailed Curriculum Concepts

Computational Thinking: Implements financial ledger tracking by evaluating individual inputs against state boundaries.

Operators & Types: Blends comparison checks with compound logical sequences (or).

Tuples & Unpacking: Extracts discrete transaction data points via sequential step unpacks.

Strings: Constructs runtime execution details via custom string combination operations.

Code Analysis: Ensures processing criteria safely catches zero or negative values immediately.

Algorithms & Pseudocode: Evaluates transaction type tags to alter internal financial totals securely.

Pattern Generation & Loops: Maps systematic list evaluations sequentially over arbitrary operation collections.

Nested Lists & Tuples: Standard linear multi-element data list packaging (List[Tuple[str, int]]).

Code Tracing Execution Log

Step	Type	Amount	Validation Logic Check	Balance State	Total Withdrawn	Log Summary Item
Init	-	-	-	10000	0	[]
1	withdraw	4000	4000 <= 10000 and 0 + 4000 <= 50000 → OK	6000	4000	"withdraw: Success"
2	deposit	2000	Direct addition → OK	8000	4000	"deposit: Success"

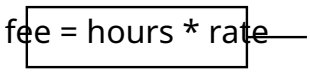
---

4. Code Block 4: calculate_parking

Output: `{'hours': 5, 'total_fee': 350}`

```
def calculate_parking(hours, vehicle_type, baseline_rate=50, penalty=0):
    fee = hours * baseline_rate
    if vehicle_type == "truck":
        fee += 100
    if hours > 24:
        fee += penalty
    return {"hours": hours, "total_fee": fee}
```

Flowchart Matrix



Detailed Curriculum Concepts

- **Computational Thinking:** Implements a layered pricing tier using standalone rule checks.
- **Operators & Types:** Multiplies base scales (*) before running compound summary updates (+=).
- **Tuples & Unpacking:** Handles straightforward scalar configurations without grouping or structures.
- **Strings:** Performs exact matches against literal string classifications.
- **Code Analysis:** Evaluates pricing formulas where consecutive conditionals accumulate flat fees sequentially.
- **Algorithms & Pseudocode:** Base calculation = duration * rate. If truck, apply flat surcharges. If day limit exceeded, apply penalties.
- **Pattern Generation & Loops:** Simple linear structural flow executing without loops.
- **Nested Lists & Tuples:** Single level value parameters.

Code Tracing Execution Log

Hours	Vehicle Type	Baseline Calculation	Truck Adjustment Check	Overnight Penalty Check	Final Fee Result
5	"truck"	5 * 50 = 250	True (+100) → 350	False (5 <= 24) → No change	**350**

---

5. Code Blocks 5 & 7: simulate_tank

Output: `{'final_level': 850, 'step': 2}`

```
def simulate_tank(capacity, initial, schedule, threshold):
    level = initial
    step_reached = -1
    index = 1
    for inflow, outflow in schedule:
        level = level + inflow - outflow
        if level > capacity: level = capacity
        if level < 0: level = 0
        if level >= threshold and step_reached == -1:
            step_reached = index
        index += 1
    return {"final_level": level, "step": step_reached}
```

Flowchart Matrix



Detailed Curriculum Concepts

Computational Thinking: Manages system state variables across dynamic input flow boundaries.

Operators & Types: Blends compound arithmetic logic alongside relational comparisons.

Tuples & Unpacking: Extracts matched delta metrics dynamically via step structures.

Strings: Standard dictionary tracking keys without external string variations.

Code Analysis: Restricts resource ranges strictly inside configured caps (0 to capacity) regardless of raw inputs.

Algorithms & Pseudocode: Iterates updates step-by-step; retains the initial matching boundary flag permanently once caught.

Pattern Generation & Loops: Linear collection scanning combined with a persistent manual tracking index.

Nested Lists & Tuples: Decodes paired collection tracks safely inside an active sequence loop.

Code Tracing Execution Log

Step Index	Inflow / Outflow	Calculated Change	Intermediate Level	Clamped Level	Threshold Check (>=700)	Stored Step Flag
Init	-	-	-	200	False	-1
1	300 / 100	+200	200 + 200 = 400	400	400 < 700 → No change	-1
2	500 / 50	+450	400 + 450 = 850	850	850 >= 700 → Triggered	**2**

---

6. Code Block 6: find_best_route

Output: `{'car': 'Sedan', 'cost': 2300.0}`

```
def find_best_route(distance, fuel_price, vehicles):
    best_car = ""
    lowest_cost = 99999999
    for name in vehicles:
        v = vehicles[name]
        fuel_needed = (distance / v["mileage"]) + v["reserve"]
        cost = fuel_needed * fuel_price
        if cost < lowest_cost:
            lowest_cost = cost
            best_car = name
    return {"car": best_car, "cost": lowest_cost}
```

Detailed Curriculum Concepts

- **Computational Thinking:** Normalizes cost parameters across multiple complex objects to identify optimal efficiency.
- **Operators & Types:** Blends math steps (/, +) with floating-point scale comparisons.
- **Tuples & Unpacking:** Iterates object keys to access child dictionary values explicitly.
- **Strings:** Tracks vehicle models as identifiers to return the optimal name.
- **Code Analysis:** Relies on a large numeric sentinel value (99999999) to initialize the minimum optimization tracking logic.
- **Algorithms & Pseudocode:** Evaluates consumption models step-by-step; keeps the vehicle with the lowest calculated operating cost.
- **Pattern Generation & Loops:** Evaluates distinct named structural entities using clean single-pass maps.
- **Nested Lists & Tuples:** Traverses nested dictionaries containing structured vehicle configuration variables.

Code Tracing Execution Log

Vehicle Name	Mileage Config	Reserve Config	Fuel Needed Calculation	Total Cost Calculation	Optimization Check (Cost < Min)	Lowest Cost State	Best Car State
Init State	-	-	-	-	-	99999999	""
SUV	10	5	(300 / 10) + 5 = 35	35 * 100 = 3500	3500 < 99999999 → True	3500	"SUV"
Sedan	15	3	(300 / 15) + 3 = 23	23 * 100 = 2300	2300 < 3500 → True	**2300**	**"Sedan"**

---

7. Code Block 8: validate_storage

Output: `{'valid': False, 'errors': ['Limit exceeded: Acid', 'Incompatible: Acid and Base']}`

```
def validate_storage(inventory, limits, incompatibles):
    violations = []
    for item in inventory:
        cat = item["cat"]
        if cat in limits and item["qty"] > limits[cat]:
            violations.append("Limit exceeded: " + cat)
    for c1, c2 in incompatibles:
        has_c1, has_c2 = False, False
        for item in inventory:
            if item["cat"] == c1: has_c1 = True
            if item["cat"] == c2: has_c2 = True
        if has_c1 and has_c2:
            violations.append("Incompatible: " + c1 + " and " + c2)
    return {"valid": len(violations) == 0, "errors": violations}
```

Detailed Curriculum Concepts

- **Computational Thinking:** Implements complex validation logic by separating capacity checks from cross-element incompatibility rules.
- **Operators & Types:** Uses logical membership operations (in, and) alongside length checks (len() == 0).
- **Tuples & Unpacking:** Automatically extracts structured pairs from the incompatibility checklist rule set.
- **Strings:** Combines dynamic alerts via concatenation string steps.
- **Code Analysis:** Evaluates safety rule compliance across lists; logs all observed structural violations.
- **Algorithms & Pseudocode:** Scans quantities against defined limits, then evaluates structural pairing exclusions across active components.
- **Pattern Generation & Loops:** Uses sequential single-pass processing followed by structural nested verification matrix passes.
- **Nested Lists & Tuples:** Coordinates operations across compound multi-type input configurations.

Code Tracing Execution Log

Phase / Check Target	Item Context	Evaluation Conditions Applied	Observed Action Taken	Resulting Violations List State
Limit Phase: Item 1	Acid (Qty: 50)	"Acid" in limits and 50 > 40 → True	Log Limit Deficit	["Limit exceeded: Acid"]
Limit Phase: Item 2	Base (Qty: 20)	"Base" in limits → False	No change	["Limit exceeded: Acid"]
Incompatibility Check	("Acid", "Base")	Inventory contains "Acid" and "Base" → True	Log Incompatibility Alert	**["Limit exceeded: Acid", "Incompatible: Acid and Base"]**